LAPORAN PRAKTIKUM

MODUL 7

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi Syarat Kelengkapan Praktikum Mata Kuliah Teknologi IOT 2

Disusun Oleh :

Nama : Muhammad Khoiru Nizar

NPM : 25782018

Jurusan : Teknologi Informasi

Program Study : Teknologi Rekaysa Internet

Kelas : 3A

Dosen Pengampu :

Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

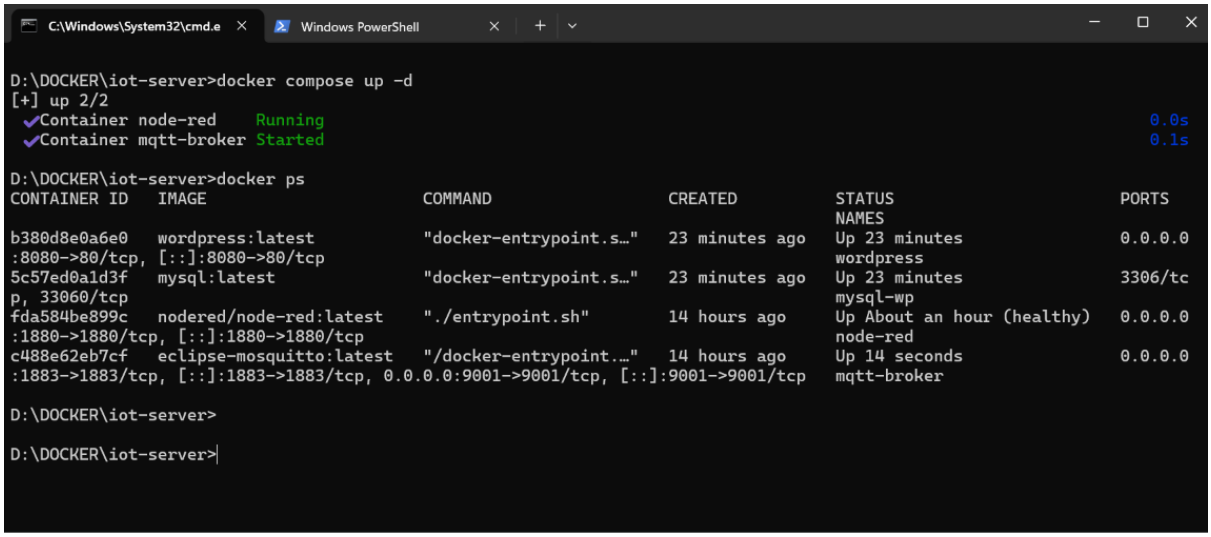
Agus Ambarwari, S.Pd.,M.Kom.

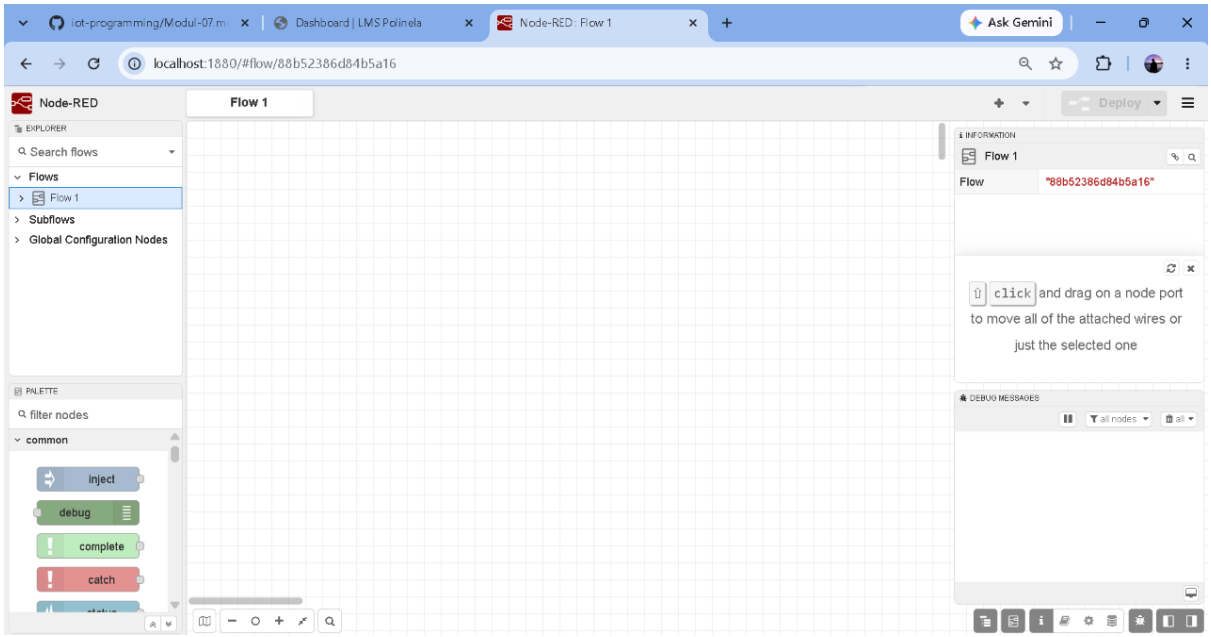
MENTRI PENDIDIKAN DAN KEBUDAYAAN KOTA BANDAR LAMPUNG POLITEKNIK NEGERI LAMPUNG

TAHUN AJARAN 2026/2027

Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.





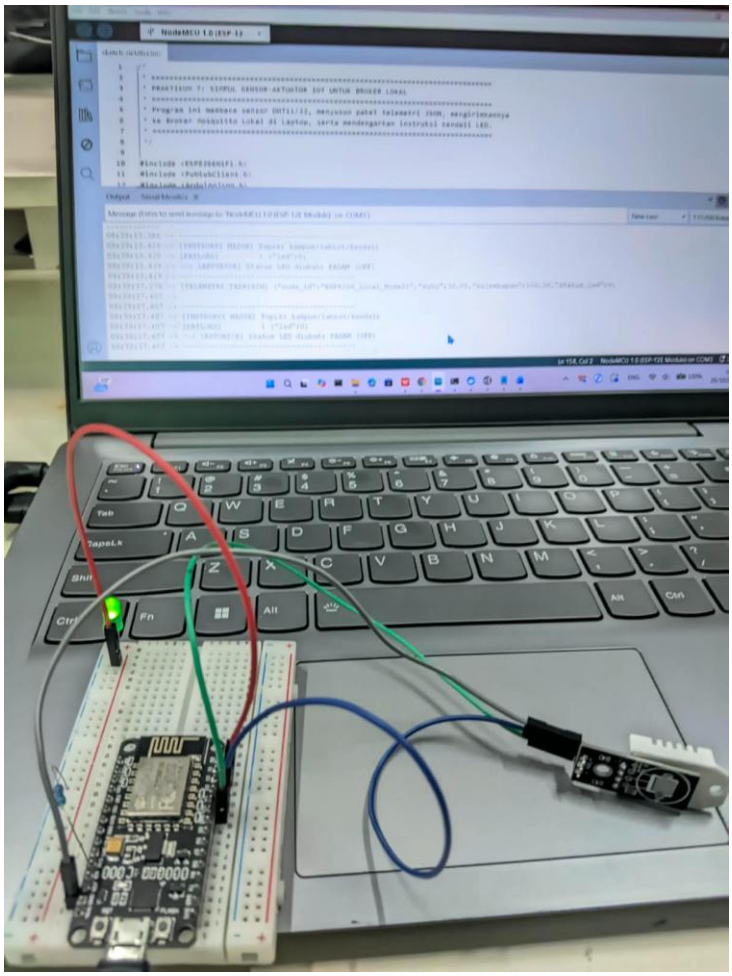
Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

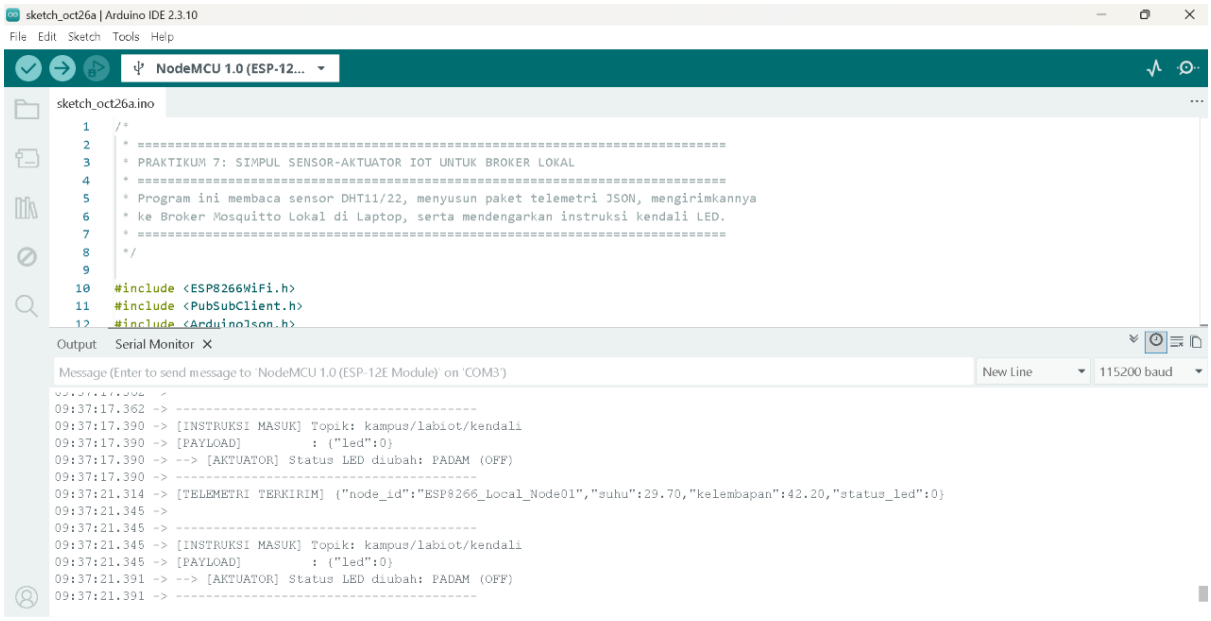
Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

Skema Rangkaian Perangkat Keras:

Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

- **Sensor DHT11/DHT22:**
    - Pin VCC →Pin 3V3 NodeMCU
    - Pin GND →Pin GND NodeMCU
    - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
    - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
    - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND




sketch_oct26a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct26a.ino
/*
* ==
* PRAKTIKUM 7: SIMPUL SENSOR-AKTUATOR IOT UNTUK BROKER LOKAL
* ==
* Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
* ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
* ==
*/
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
115200 baud
09:37:17.362 -> ----------
09:37:17.390 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
09:37:17.390 -> [PAYLOAD] : {"led":0}
09:37:17.390 -> --> [AKTUATOR] Status LED diubah: PADAM (OFF)
09:37:17.390 -> ----------
09:37:21.314 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":29.70,"kelembapan":42.20,"status_led":0}
09:37:21.345 ->
09:37:21.345 -> ----------
09:37:21.345 -> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
09:37:21.345 -> [PAYLOAD] : {"led":0}
09:37:21.391 -> --> [AKTUATOR] Status LED diubah: PADAM (OFF)
09:37:21.391 -> ----------

Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
    - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
    - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
        - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
        - **Name:** Mosquitto Local
        - **Server:** Ketik **mqtt**
          (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
        - **Port:** 1883
        - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
    - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
    - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
    - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
    - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
    - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**
    - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.

- Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.

6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang (🐞) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:

     kampus/labiot/telemetri : msg.payload : Object

     {

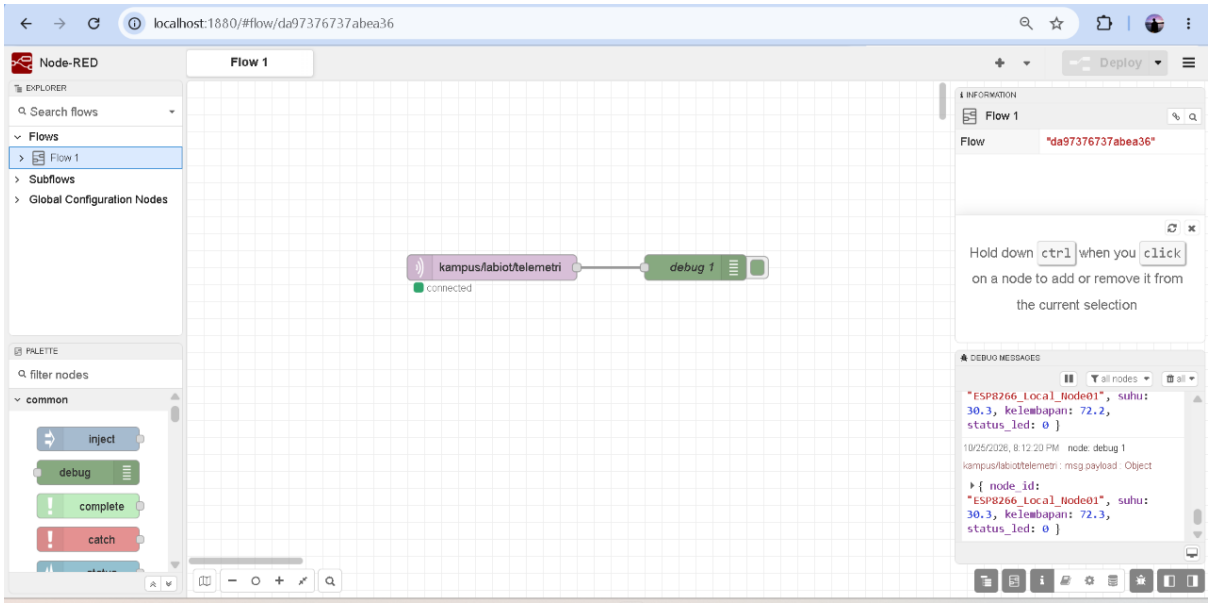
     node_id: "ESP8266_Local_Node01",

     suhu: 29.8,

     kelembapan: 62.4,

     status_led: 0

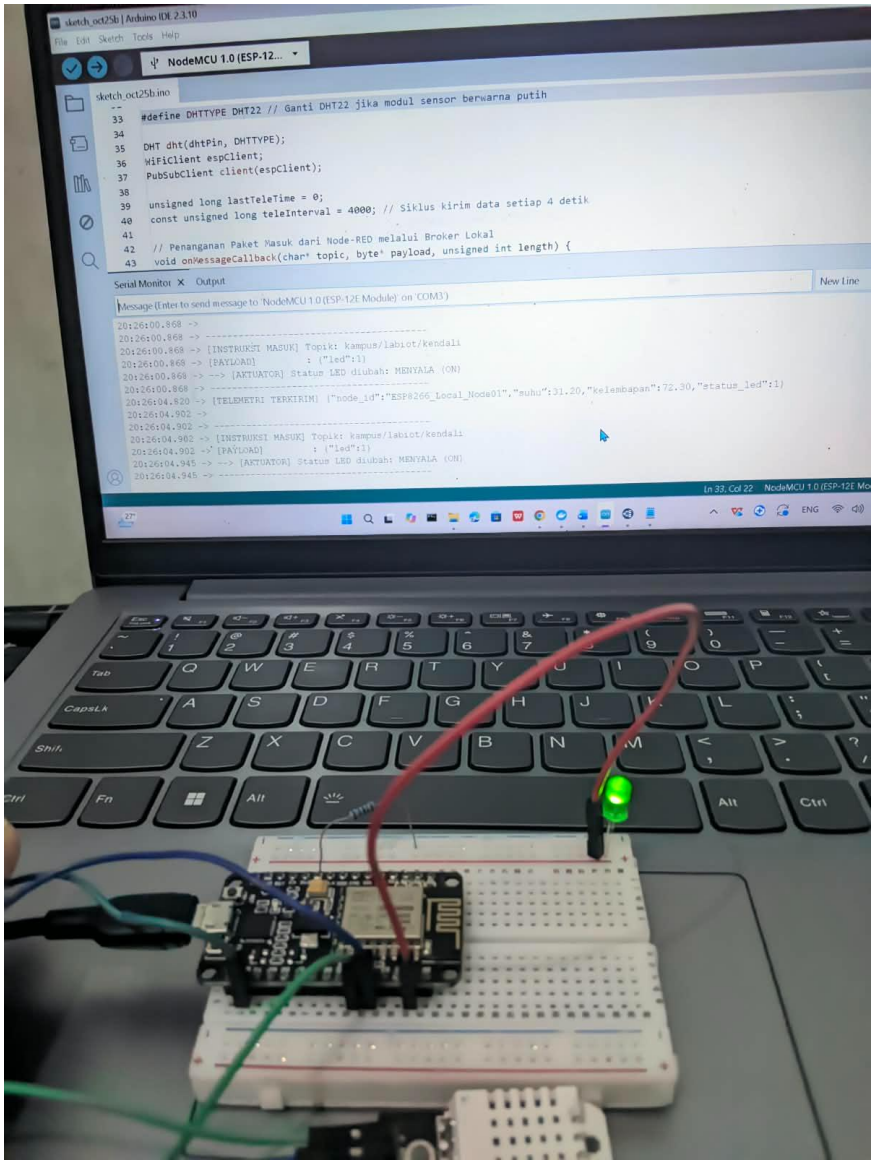
     }
   - Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.

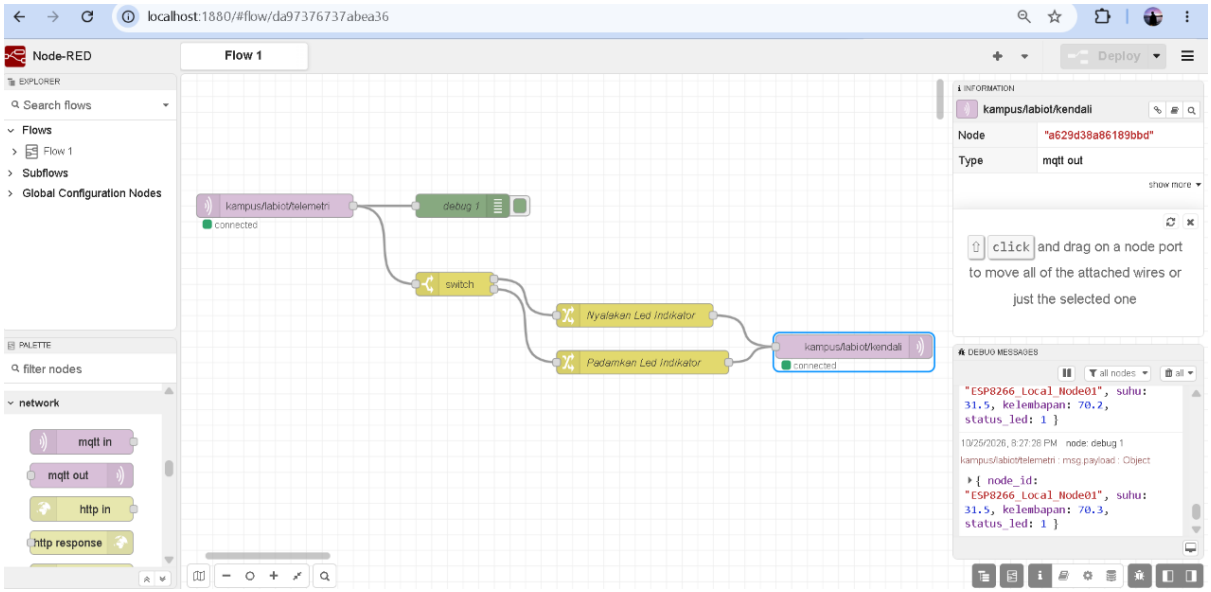


Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.





Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Jawab: Karena localhost di dalam kontainer Node-RED merujuk ke kontainer itu sendiri, bukan ke laptop atau kontainer Mosquitto (mqtt-broker).

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Jawab : Jaringan virtual (*Docker Bridge*) memiliki server DNS internal yang otomatis menerjemahkan nama layanan (seperti mqtt) langsung ke IP internal kontainer broker, sehingga Node-RED bisa terhubung tanpa menggunakan IP statis.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  Jawab: Karena NodeMCU berada di subnet yang berbeda (192.168.43.x dari hotspot HP) dengan laptop (192.168.1.x), sehingga kedua perangkat tidak bisa saling berkomunikasi secara langsung.

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

  Jawab: Saya harus menyambungkan laptop dan NodeMCU ke jaringan Wi-Fi/hotspot yang persis sama agar berada dalam satu rentang subnet yang seirama.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas $31.0°C$(misal: $31.05°C \rightarrow 30.95°C \rightarrow 31.02°C$), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

  - LED baru menyala jika suhu $> 31.5°C$(msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga $< 30.0°C$(msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
let suhu = msg.payload.suhu;

let statusLed = context.get('statusLed') || 0;


if (suhu > 31.5) {
   statusLed = 1; // Nyalakan LED jika panas
} else if (suhu < 30.0) {
   statusLed = 0; // Matikan LED jika dingin
}
```

```
msg.payload = { "led": statusLed };

context.set('statusLed', statusLed);

return msg;
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  Jawab: Seluruh alur *flow* dan konfigurasi yang telah saya buat di Node-RED akan hilang total karena kembali ke kondisi awal (*fresh state*).

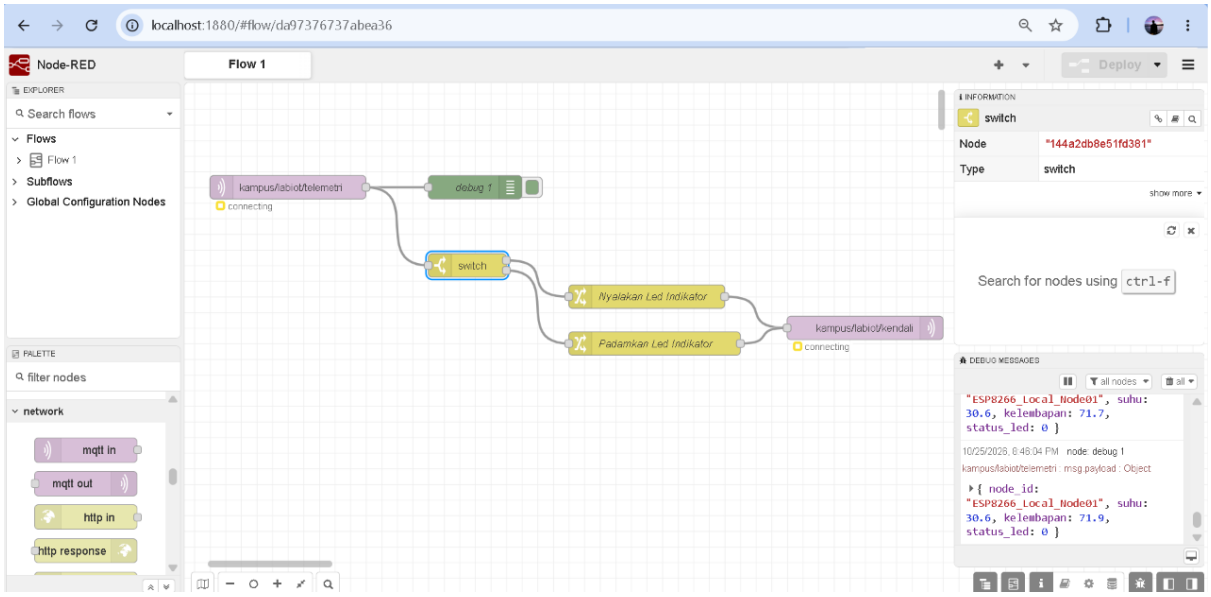
  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

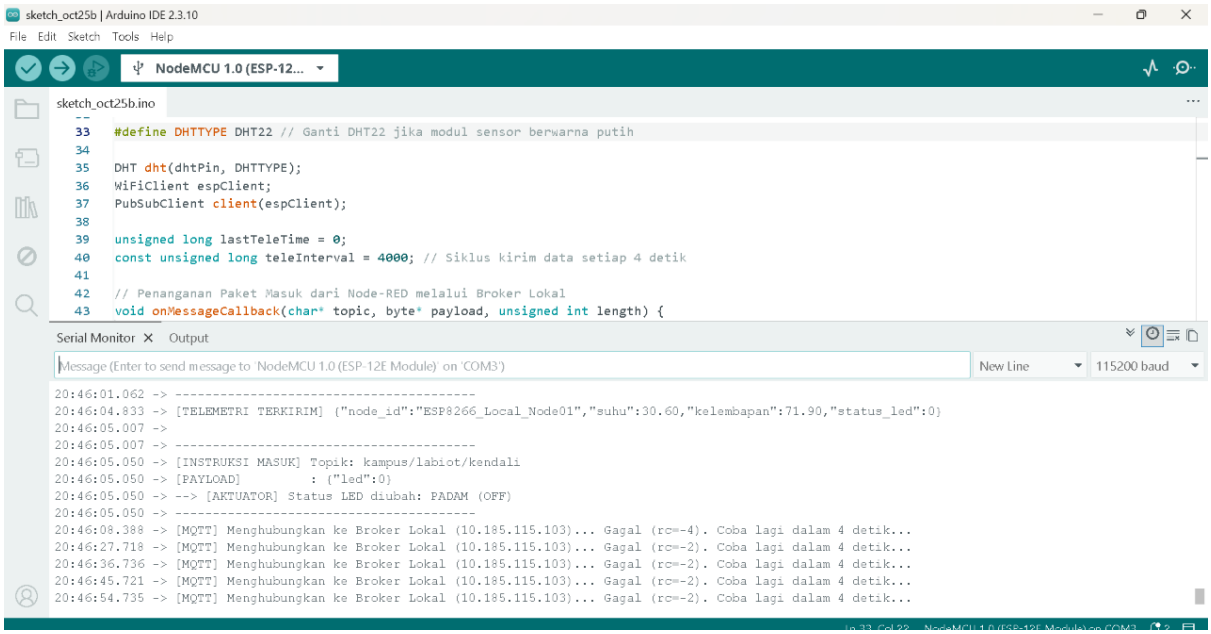
  Jawab:

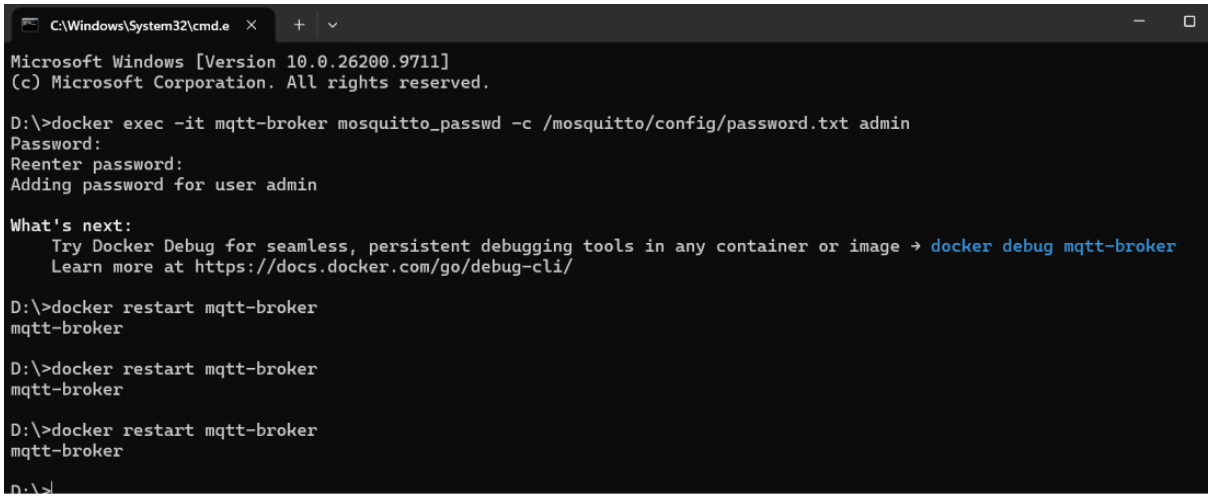
- **Bind Mount:** Mengarahkan langsung ke folder spesifik di komputer host (misal ./nodered:/data).
- **Named Volume:** Penyimpanan dikelola secara otomatis di dalam area khusus oleh Docker tanpa perlu menentukan folder host secara manual.

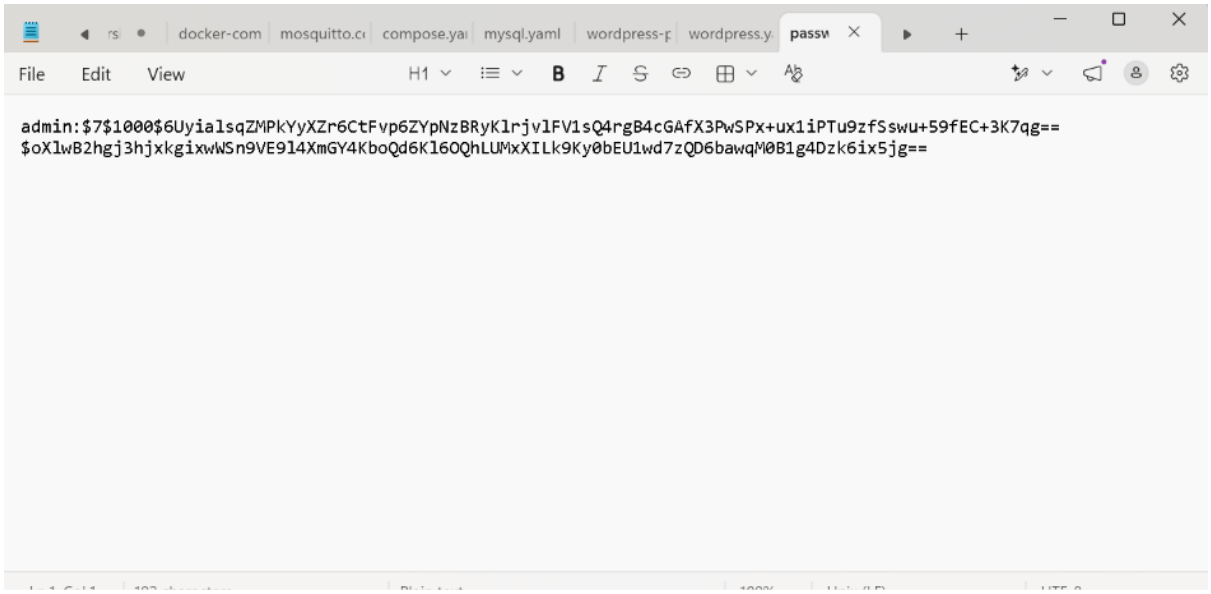
Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).






C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.
D:\>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin
What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
Learn more at https://docs.docker.com/go/debug-cli/
D:\>docker restart mqtt-broker
mqtt-broker
D:\>docker restart mqtt-broker
mqtt-broker
D:\>docker restart mqtt-broker
mqtt-broker


File
Edit
View
admin:$7$1000$6UyialsqZMPkYyXZr6CtFvp6ZYpNzBRyKlrjvlFV1sQ4rgB4cGAfX3PwSPx+ux1iPTu9zfSswu+59fEC+3K7qg==
$oXlwB2hgj3hjxkgixwWSn9VE914XmGY4KboQd6Kl6OQhLUMxXILk9Ky0bEU1wd7zQD6bawqM0B1g4Dzk6ix5jg==